



University of Dhaka

Department of Computer Science and Engineering

CSE 3111: Computer Networking Lab

Lab Report - 1

Design of a Chat Application

Using Multi-Threaded Socket Programming

Lab Group: A (ODD)

H.M. Mehedi Hasan (Roll: 13)

MD. Abu Bakar Siddique (Roll: 47)

Submission Date: September 17, 2025

Contents

1	Introduction	2
2	Objectives	2
3	Design Details	2
3.1	Implementation Process	2
3.2	Implementation Flowcharts	3
4	Implementation	5
4.1	Server Implementation	5
4.2	Client Implementation	8
5	Result Analysis	8
5.1	Server Output	8
5.2	Client 1 Output	10
5.3	Client 2 Output	10
6	Discussion	11
6.1	Basic Socket Programming vs. Multi-threaded Socket Programming . . .	11
6.2	Drawbacks of Basic Socket Programming	11
6.3	How Multi-threading Overcomes These Drawbacks	12
6.4	Learnings and Experience	12
6.5	Difficulties Faced	12

1 Introduction

Socket programming is a core networking technique that enables communication between processes over a network. Using *sockets*, applications can send and receive data reliably via transport layer protocols like TCP and UDP. It is widely used in networked applications such as web servers, multiplayer games, and chat systems, where efficient data exchange is critical.

Multi-threaded socket programming enhances basic socket communication by allowing a server to handle multiple clients simultaneously. Each client connection is managed by a dedicated thread, enabling concurrent message exchange and preventing one client from blocking others. This approach is essential for maintaining real-time responsiveness in interactive applications.

Necessity in chat applications:

Chat applications require continuous, bidirectional communication between multiple users. A single-threaded server can serve only one client at a time, causing delays and poor user experience. Multi-threading allows the server to manage multiple clients efficiently, broadcast messages instantly, and provide a scalable, responsive, and seamless interactive environment.

2 Objectives

1. To gain a thorough understanding of socket programming and the role of multi-threading in networked applications.
2. To design and implement a multi-threaded chat server capable of handling multiple clients simultaneously.
3. To enable continuous, bidirectional communication, allowing clients to send and receive messages asynchronously in real time.

3 Design Details

3.1 Implementation Process

1. Server Setup:

- (a) The server creates a `ServerSocket` on port 12345.
- (b) It continuously listens for incoming client connections in a separate thread.
- (c) Maintains a mapping of connected clients using unique client IDs and their output streams.
- (d) Provides a console interface for server commands:
 - `shutdown` to stop the server and terminate all connections.
 - `C<clientId>: <message>` to send a message to a specific client.

2. Client Handling:

- (a) Each new client connection triggers the creation of a dedicated `ClientHandler` thread.
- (b) The thread manages:
 - Receiving messages from the client.
 - Sending messages to the client via its stored `PrintWriter`.
 - Handling client exit gracefully when "exit" is received.
- (c) All client output streams are stored in a shared `HashMap` for targeted messaging.

3. Client Setup:

- (a) The client establishes a connection to the server at `localhost:12345`.
- (b) The client operates with two concurrent tasks:
 - One task continuously listens for messages from the server and displays them to the user.
 - The other task waits for user input and forwards it to the server.
- (c) Entering "exit" ends the session and closes the connection.

3.2 Implementation Flowcharts

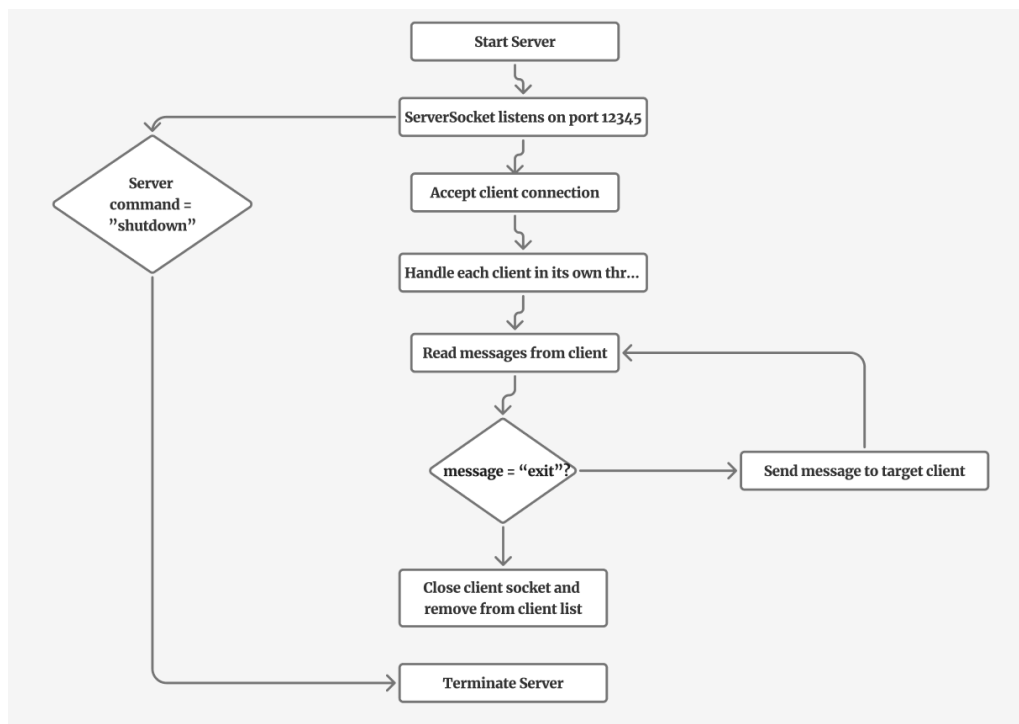


Figure 1: Implementation flowchart of the Server

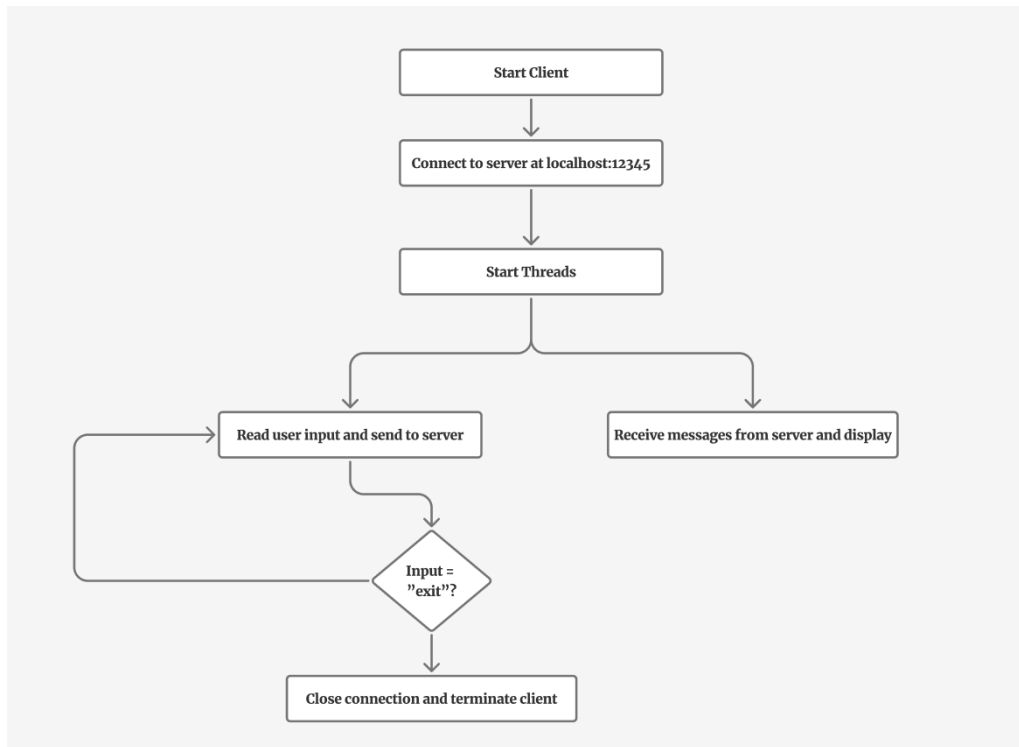


Figure 2: Implementation flowchart of the Client

4 Implementation

4.1 Server Implementation

```
import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer_13_47{
    private static int clientIdCounter = 1;
    private static HashMap<Integer, PrintWriter> clientOutputs = new HashMap<>();

    Run | Debug
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(port:12345);
            System.out.println(x:"Chat Server started on port 12345... waiting for clients to connect");

            new Thread(() -> {
                while (true) {
                    try {
                        Socket socket = serverSocket.accept();
                        int clientId = clientIdCounter++;
                        ClientHandler handler = new ClientHandler(socket, clientId);
                        handler.start();
                    } catch (IOException e) {
                        System.out.println("Error accepting client connection: " + e.getMessage());
                        break;
                    }
                }
            }).start();

            Scanner scanner = new Scanner(System.in);
            System.out.println(x:"Server is running. Commands:");
            System.out.println(x:"- Type 'shutdown' to stop the server");
            System.out.println(x:"- Type 'C<number>: <message>' to send a message to a specific client");
            System.out.println(x:" Example: C1: Hello there!");

            while (true) {
                String input = scanner.nextLine();
                if (input.equalsIgnoreCase(anotherString:"shutdown")) {
                    System.out.println(x:"Server shutting down...");
                    break;
                }
                if (input.startsWith(prefix:"C") && input.contains(s:":")) {
                    String[] parts = input.split(regex:":", limit:2);
                    try {
                        int clientId = Integer.parseInt(parts[0].substring(beginIndex:1));
                        String message = parts[1].trim();
                        PrintWriter clientOutput = clientOutputs.get(clientId);
```

Figure 3: Server Code - Part 1

```
        if (clientOutput != null) {
            clientOutput.println("Server: " + message);
            clientOutput.flush();
            System.out.println("Message sent to Client " + clientId);
        } else {
            System.out.println("Error: No client found with ID " + clientId);
        }
    } catch (NumberFormatException e) {
        System.out.println(x:"Invalid client ID format.");
        continue;
    }
} else {
    System.out.println(x:"Invalid format. Use: C<number>: <message>");
    System.out.println(x:"Example: C1: Hello!");
}
}

serverSocket.close();
scanner.close();

} catch (IOException e) {
    System.out.println("Server error: " + e.getMessage());
}
}

static class ClientHandler extends Thread {
    private Socket clientSocket;
    private int clientId;

    public ClientHandler(Socket socket, int clientId) {
        this.clientSocket = socket;
        this.clientId = clientId;
    }

    public void run() {
        try {
            BufferedReader input = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
            PrintWriter output = new PrintWriter(clientSocket.getOutputStream(), autoFlush:true);
            clientOutputs.put(clientId, output);
            System.out.println("Client " + clientId + " connected successfully.");
            output.println("Welcome! You are Client " + clientId);
```

Figure 4: Server Code - Part 2

```
static class ClientHandler extends Thread {
    private Socket clientSocket;
    private int clientId;

    public ClientHandler(Socket socket, int clientId) {
        this.clientSocket = socket;
        this.clientId = clientId;
    }

    public void run() {
        try {
            BufferedReader input = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream())
            );
            PrintWriter output = new PrintWriter(clientSocket.getOutputStream(), autoFlush:true);
            clientOutputs.put(clientId, output);
            System.out.println("Client " + clientId + " connected successfully.");
            output.println("Welcome! You are Client " + clientId);
            output.println(x:"Type 'exit' to disconnect from the server.");
            String clientMessage;
            while ((clientMessage = input.readLine()) != null) {
                if (clientMessage.equalsIgnoreCase(anotherString:"exit")) {
                    System.out.println("Client " + clientId + " disconnected.");
                    break;
                }
                System.out.println("Client " + clientId + " says: " + clientMessage);
            }
            clientOutputs.remove(clientId);
            clientSocket.close();
        } catch (IOException e) {
            System.out.println("Error handling Client " + clientId + ": " + e.getMessage());
            clientOutputs.remove(clientId);
        }
    }
}
```

Figure 5: Server Code - Part 3

4.2 Client Implementation

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ChatClient_13_47{
    Run | Debug
    public static void main(String[] args) {
        try {
            Socket socket = new Socket(host:"localhost", port:12345);
            System.out.println(x:"Connected to server!");

            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush:true);

            new Thread(() -> {
                try {
                    String response;
                    while ((response = in.readLine()) != null) {
                        System.out.println(response);
                    }
                } catch (IOException e) {
                    System.out.println(x:"Connection closed.");
                }
            }).start();

            Scanner sc = new Scanner(System.in);
            while (true) {
                String msg = sc.nextLine();
                out.println(msg);
                if (msg.equalsIgnoreCase(anotherString:"exit")) {
                    break;
                }
            }

            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Figure 6: Client Code

5 Result Analysis

The chat application was tested with multiple clients to validate its functionality. The results confirm that the system successfully implements multi-threaded socket programming, allowing concurrent client connections, real-time communication, and controlled server-side management. Below is a detailed analysis of the outputs from both the server and client terminals.

5.1 Server Output

On starting the server, the console displays:

- A confirmation that the server is running and listening on port 12345.
- A list of available commands, such as **shutdown** to terminate the server and **C<number>: <message>** to send direct messages to a specific client.

- Notifications when clients successfully connect to the server (e.g., `Client 1 connected successfully`, `Client 2 connected successfully`).
- Messages received from clients, displayed in the format: `Client <id> says: <message>`.
- Confirmation of server-to-client communication, e.g., `Message sent to Client 1`.
- Proper handling of client disconnections when users type `exit`.
- Graceful termination when the `shutdown` command is issued, closing the server socket and stopping new connections.

This demonstrates that the server:

1. Handles multiple clients concurrently using separate threads.
2. Logs all client activities (connections, messages, disconnections).
3. Allows the server operator to send answers or instructions to clients individually.
4. Provides a controlled exit mechanism for shutting down the entire system.

```
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> javac ChatServer_13_47.java
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> java ChatServer_13_47
Chat Server started on port 12345... waiting for clients to connect
Server is running. Commands:
- Type 'shutdown' to stop the server
- Type 'C<number>: <message>' to send a message to a specific client
  Example: C1: Hello there!
Client 1 connected successfully.
Client 2 connected successfully.
Client 1 says: What is 2+3?
Client 2 says: What is http full form?
C1: 5
Message sent to Client 1
C2: Hypertext Transfer Protocol
Message sent to Client 2
Client 1 says: What is AI?
Client 1 says: What is ML?
Client 2 says: Capital of Spain?
C1: Artificial Intelligence
Message sent to Client 1
C1: Machine Learning
Message sent to Client 1
C2: Madrid
Message sent to Client 2
Client 1 disconnected.
Client 2 disconnected.
shutdown
Server shutting down...
Error accepting client connection: Socket closed
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment>
```

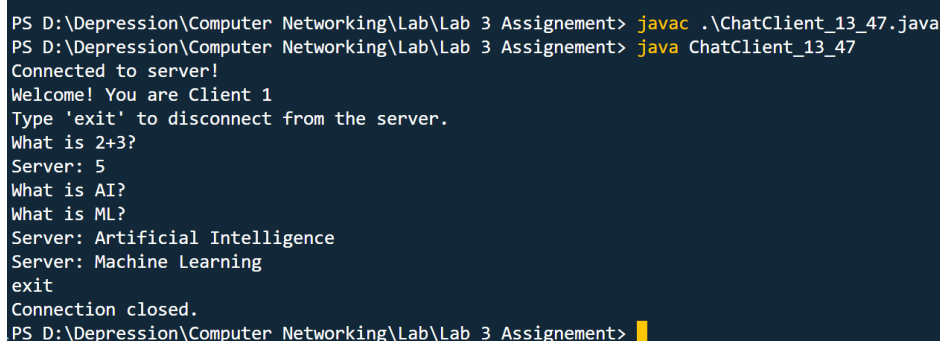
Figure 7: Server Output showing initialization, client connections, message handling, and shutdown.

5.2 Client 1 Output

Client 1 demonstrates the following:

- On connection, the client receives a welcome message and instructions from the server.
- Messages typed by the user (e.g., `What is 2+3?`, `What is AI?`) are sent to the server.
- Server responses are displayed back in the client terminal (e.g., `Server: 5`, `Server: Artificial Intelligence`).
- The client terminates the session by typing `exit`, after which the connection closes gracefully.

This confirms that Client 1 can both send queries and receive targeted replies from the server in real time.



```
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> javac .\ChatClient_13_47.java
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> java ChatClient_13_47
Connected to server!
Welcome! You are Client 1
Type 'exit' to disconnect from the server.
What is 2+3?
Server: 5
What is AI?
What is ML?
Server: Artificial Intelligence
Server: Machine Learning
exit
Connection closed.
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> 
```

Figure 8: Client 1 Output showing query messages and server responses.

5.3 Client 2 Output

Similarly, Client 2 output demonstrates:

- Successful connection to the server with a welcome message.
- Ability to send independent queries (e.g., `What is http full form?`, `Capital of Spain?`).
- Correct responses received from the server (`Hypertext Transfer Protocol`, `Madrid`).
- Graceful disconnection after typing `exit`.

This shows that multiple clients can operate simultaneously without interfering with one another, and the server responds accurately to each.

```
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> java ChatClient_13_47
Connected to server!
Welcome! You are Client 2
Type 'exit' to disconnect from the server.
What is http full form?
Server: Hypertext Transfer Protocol
Capital of Spain?
Server: Madrid
exit
Connection closed.
PS D:\Depression\Computer Networking\Lab\Lab 3 Assignment> █
```

Figure 9: Client 2 Output showing independent queries and corresponding server responses.

6 Discussion

This task provided practical insights into the use of socket programming and multi-threading in network applications. The following points summarize the key observations and comparisons:

6.1 Basic Socket Programming vs. Multi-threaded Socket Programming

- **Basic Socket Programming:** In a simple socket server, communication is handled sequentially. The server can only interact with one client at a time. When a client is being served, all other clients must wait, leading to blocking and delays. This design is simple but highly inefficient for real-time applications such as chat systems.
- **Multi-threaded Socket Programming:** By introducing multi-threading, each client connection is managed in a separate thread. This allows the server to handle multiple clients concurrently, enabling parallel message exchanges. As a result, one client's activities do not block or delay others, which makes the system scalable and responsive.

6.2 Drawbacks of Basic Socket Programming

1. Only one client can be served at a time.
2. Long-running operations block other clients.
3. Poor scalability for applications with many users.
4. Unsuitable for real-time, interactive systems such as chat applications.

6.3 How Multi-threading Overcomes These Drawbacks

1. Each client is handled by an independent thread, avoiding blocking.
2. The server can broadcast or send targeted messages simultaneously.
3. System responsiveness improves significantly, even with multiple clients.
4. The program design becomes closer to real-world chat platforms, where users expect real-time interactions.

6.4 Learnings and Experience

Through this task, several important concepts were reinforced:

- Understanding of socket communication using Java's `ServerSocket` and `Socket` classes.
- Practical implementation of multi-threading to manage concurrent tasks.
- Techniques to send targeted messages to specific clients, enhancing flexibility.
- Handling graceful termination of clients and the server using commands such as `exit` and `shutdown`.

6.5 Difficulties Faced

During the implementation, a few challenges were encountered:

- Ensuring thread safety while managing shared resources such as the client list.
- Designing a clear communication protocol for both server-to-client and client-to-server interactions.
- Debugging concurrent processes, as multiple threads made the execution flow harder to trace.
- Managing clean disconnection of clients to avoid resource leaks.

Despite these challenges, the implementation was successful, and the project provided hands-on experience with real-world issues in network programming.